

Apprentissage Profond (I-ILIA-029) -- Travaux pratiques

TP 05 : CNN et Transfer Learning pour la classification d'objets dans un chantier ferroviaire

L'objectif de ce TP est de développer un module de vidéo surveillance qui consiste à classer des images représentant les objets (rails de train, train, pelleuseuse) et ouvriers dans un chantier ferroviaire. Ceci permettra de sécuriser les chantiers ferroviaires grâce à la détection et localisation de tous les objets dangereux sur chantier. Plus concrètement, le modèle devra prendre en entrée une image couleur avant de la traiter par un réseau de neurones profonds pour identifier le label de l'image parmi 4 classes possibles (Fig. 1). Pour réaliser ce TP, nous vous proposons d'exploiter les réseaux de neurones convolutionnels (CNN) ainsi que le principe du Transfer Learning.



Figure 1 : exemples d'images représentant les classes à identifier avec un CNN

- **Question 1** « Vérification du matériel » : importer le notebook « *TP5_I_ILIA_029_Input_2025.ipynb* » et vérifier que le GPU est bien sélectionné ;
- **Instruction 1** « Installation » : installer la librairie Pytorch Lightning ;
- **Question 2** : connecter votre notebook à la plateforme **WanDB** ;
- **Instruction 2** : télécharger du jeu de données d'entraînement : **railway-construction-50**
 - BD railway-construction-50 : <https://nextcloud.ig.umons.ac.be/s/zBCAj6GFjQjA3fz/download/railway-construction-50.zip>

Note 1 : cette base est représentée par les quatre classes ou chaque classe dispose de 50 images.

- **Instruction 3** : « Importation des libraires » : importer les libraires nécessaires ;
- **Instruction 4** : définir les paramètres d'entraînement "Hyper-paramètres" ;
- **Question 3** : analyser attentivement la classe « **Model** » avant de la compléter pour faire un transfert d'apprentissage à partir du modèle pré-entraîné **VGG16** (fonction d'erreur : CrossEntropyLoss).
- **Instruction 5** : analyser et exécuter la fonction d'affichage des images ;
- **Question 4** : afficher quelques images de chaque classe ;

- **Instruction 6** : analyser et exécuter la fonction de division des données et création des tenseurs/dataloaders
- **Question 5** : créer les data loaders (train, val, test) avec la fonction « *create_data_loader* »
- **Question 6** : compléter le code (SGD avec *lr='0.01'*) et lancer l'entraînement
- **Instruction 7** : analyser et exécuter fonction de visualisation des courbes d'entraînement avec plot ;
- **Question 7** : afficher vos courbes d'entraînement avec la fonction « *plot_metrics* »
- **Question 8** : vérifier/interpréter les courbes d'entraînement de votre modèle via Wandb ou plot ;
- **Question 9** : évaluer votre modèle avec les données de test
- **Instruction 8** : exporter le modèle en format torchscript pour la soumission Kaggle
- **Instruction 9** : télécharger la base de données test non labellisée
 - BD de test sans labels: https://nextcloud.ig.umons.ac.be/s/imNGnwZtSEnYJNL/download/test_no_classes.zip
- **Instruction 10** : normaliser et transformer les données de test ;
- **Instruction 11** : analyser/exécuter la fonction de génération des prédictions pour le challenge Kaggle
- **Question 10** : appliquer cette fonction pour générer les prédictions csv à soumettre via Kaggle
- **Question 10b** : soumettre vos prédictions au challenge kaggle via ce [lien](#) ;
- **Question 11** : modifier le code pour appliquer l'*earlystopping* et *checkpoint* (Chap. 5)
- **Question 12** : modifier le code pour calculer la matrice de confusion en phase de test

```
y_hat_softmax = F.softmax(y_hat, dim=1)
test_acc = multiclass_accuracy( preds=y_hat_softmax, target=y, num_classes=self.num_classes)
self.confusion_matrix.update(y_hat.argmax(dim=1), y)
```

```
fig_, ax_ = self.confusion_matrix.plot()
plt.show()
self.confusion_matrix.reset()
```

Facultatif :

- **Question 13** : refaire le même travail avec des base de données plus grandes et accessible via ces liens
 - BD2 (100 images par classe) : <https://nextcloud.ig.umons.ac.be/s/kyN88LnfRrS9rri/download/railway-construction-100.zip>
 - BD2 (>100 images par classe) : <https://nextcloud.ig.umons.ac.be/s/SAFseyDzbyPXcj2/download/railway-construction-big.zip>
 - **Question 14** : tester d'autres architectures (EfficientNet_v2_s, MoblieNet, ResNet152, etc.)
- **Note 2** : Le [site](#) Kaggle permettra d'évaluer vos modèles et les classer par ordre décroissant de précision.
 - **Note 3** : vous avez le libre de choix d'adapter/augmenter la base de données d'entraînement.
 - **Note 4** : l'énoncé du mini-projet sera une continuité de ce TP intégrant des notions Vit (TP6) et XAI (TP7).